

Room DB — Restaurante & Platos

Dos enfoques: @Embedded vs Tabla Intermedia (N:M)

Room no admite listas directamente en una entidad. Este documento muestra dos soluciones según el caso de uso: embeber campos fijos con @Embedded o usar una **tabla intermedia** para relaciones variables o N:M.

Criterio	@Embedded	Tabla Intermedia (N:M)
Número de platos	Fijo y conocido (ej. 3)	Variable / ilimitado
Platos compartidos entre restaurantes	No	Sí
Consulta independiente de platos	Difícil	Fácil con JOIN
Complejidad de código	Baja	Media
Escalabilidad	Baja	Alta

PARTE 1 — Enfoque con @Embedded

Cuándo usarlo: Cuando cada restaurante tiene siempre exactamente N platos (ej. 3), esos platos son exclusivos del restaurante y no necesitas buscarlos de forma independiente. Room almacena los campos del objeto embebido como columnas directas de la tabla padre.

Bug importante: En AppDataBase tienes **PlatoDao::class** en lugar de **Plato::class** — esto causará error de compilación. Corrígelo. Además, si Plato tiene @PrimaryKey y la usas como @Embedded, esa PK no aplica; considera usar un data class sin @Entity para el modelo embebido.

1. Modelo Plato (sin @Entity para uso embebido)

```
// PlatoEmbebido.kt – SIN @Entity, solo data class
data class PlatoEmbebido( val nombre: String )
```

2. Entidad Restaurante con @Embedded

```
// Restaurante.kt @Entity(tableName = "Restaurante") data class Restaurante(
@PrimaryKey(autoGenerate = true) val id: Long = 0, val nombre: String, val telefono: String, val
foto: Int, val latitud: Double, val longitud: Double, @Embedded(prefix = "pla1_") val plato1:
PlatoEmbebido, @Embedded(prefix = "pla2_") val plato2: PlatoEmbebido, @Embedded(prefix =
"pla3_") val plato3: PlatoEmbebido ) // Resultado en BD: tabla Restaurante con columnas // id,
nombre, telefono, foto, latitud, longitud, // pla1_nombre, pla2_nombre, pla3_nombre
```

3. DAO

```
// RestauranteDao.kt @Dao interface RestauranteDao { @Insert fun insertar(restaurante:
Restaurante): Long @Update fun actualizar(restaurante: Restaurante) @Delete fun
eliminar(restaurante: Restaurante) @Query("SELECT * FROM Restaurante") fun getAll():
LiveData<List<Restaurante>> @Query("SELECT * FROM Restaurante WHERE id = :id") fun
getById(id: Long): Restaurante? }
```

4. Base de datos (corregido)

```
// AppDataBase.kt @Database(entities = [Restaurante::class], version = 1, exportSchema = false)
abstract class AppDataBase : RoomDatabase() { abstract fun restauranteDao(): RestauranteDao
companion object { @Volatile private var INSTANCE: AppDataBase? = null fun getDatabase(context:
Context): AppDataBase { return INSTANCE ?: synchronized(this) { val instance =
Room.databaseBuilder( context.applicationContext, AppDataBase::class.java, "restaurante_db" )
.allowMainThreadQueries() .build() INSTANCE = instance instance } } }
```

5. Repository

```
// RestauranteRepository.kt class RestauranteRepository(private val dao: RestauranteDao) { fun
insertar(r: Restaurante): Long = dao.insertar(r) fun actualizar(r: Restaurante) =
dao.actualizar(r) fun eliminar(r: Restaurante) = dao.eliminar(r) fun listaTodo():
LiveData<List<Restaurante>> = dao.getAll() fun getById(id: Long): Restaurante? =
dao.getById(id) }
```

6. ViewModel

```
// RestauranteViewModel.kt class RestauranteViewModel(application: Application) :
AndroidViewModel(application) { private val repository: RestauranteRepository val
listaRestaurantes: LiveData<List<Restaurante>> init { val dao =
AppDataBase.getDatabase(application).restauranteDao() repository = RestauranteRepository(dao)
listaRestaurantes = repository.listaTodo() } fun insertar(r: Restaurante): Long =
repository.insertar(r) fun actualizar(r: Restaurante) = repository.actualizar(r) fun eliminar(r:
Restaurante) = repository.eliminar(r) fun getById(id: Long): Restaurante? =
repository.getById(id) }
```

PARTE 2 — Enfoque con Tabla Intermedia (N:M)

Cuándo usarlo: Cuando el número de platos es variable, un plato puede pertenecer a varios restaurantes, o necesitas consultar/filtrar platos de forma independiente. Se crea una tabla intermedia **RestaurantePlato** con las FK de ambas tablas.

Esquema de tablas



1. Entidad Plato (independiente)

```
// Plato.kt @Entity(tableName = "Plato") data class Plato( @PrimaryKey(autoGenerate = true) val id: Long = 0, val nombre: String, val precio: Double = 0.0, val descripcion: String = "" )
```

2. Entidad Restaurante (sin platos)

```
// Restaurante.kt @Entity(tableName = "Restaurante") data class Restaurante( @PrimaryKey(autoGenerate = true) val id: Long = 0, val nombre: String, val telefono: String, val foto: Int, val latitud: Double, val longitud: Double )
```

3. Tabla intermedia RestaurantePlato

```
// RestaurantePlato.kt – tabla de union N:M @Entity( tableName = "RestaurantePlato", primaryKeys = ["restaurantId", "platoId"], // PK compuesta foreignKeys = [ ForeignKey( entity = Restaurante::class, parentColumns = ["id"], childColumns = ["restaurantId"], onDelete = ForeignKey.CASCADE // Si borras restaurante, borra sus platos ), ForeignKey( entity = Plato::class, parentColumns = ["id"], childColumns = ["platoId"], onDelete = ForeignKey.CASCADE ) ], indices = [Index("platoId")] // Indice para FK secundaria ) data class RestaurantePlato( val restaurantId: Long, val platoId: Long )
```

4. Data class para la relacion (con @Relation)

```
// RestauranteConPlatos.kt – NO es @Entity, solo para consultas data class RestauranteConPlatos( @Embedded val restaurante: Restaurante, @Relation( parentColumn = "id", entityColumn = "id", associateBy = Junction( value = RestaurantePlato::class, parentColumn = "restaurantId", entityColumn = "platoId" ) ) val platos: List<Plato> )
```

5. DAOs

```
// RestauranteDao.kt @Dao interface RestauranteDao { @Insert fun insertar(r: Restaurante): Long @Update fun actualizar(r: Restaurante) @Delete fun eliminar(r: Restaurante) @Query("SELECT * FROM Restaurante") fun getAll(): LiveData<List<Restaurante>> @Query("SELECT * FROM Restaurante WHERE id = :id") fun getById(id: Long): Restaurante? // Consulta con JOIN – devuelve restaurante + lista de platos @Transaction @Query("SELECT * FROM Restaurante") fun getAllConPlatos(): LiveData<List<RestauranteConPlatos>> @Transaction @Query("SELECT * FROM Restaurante WHERE id = :id") fun getConPlatosById(id: Long): RestauranteConPlatos? } // PlatoDao.kt @Dao interface PlatoDao { @Insert fun insertar(plato: Plato): Long @Update fun actualizar(plato: Plato) @Delete fun eliminar(plato: Plato) @Query("SELECT * FROM Plato") fun getAll(): LiveData<List<Plato>> @Query("SELECT * FROM Plato WHERE id = :id") fun getById(id: Long): Plato? } // RestaurantePlatoDao.kt @Dao interface RestaurantePlatoDao { @Insert fun insertar(rel: RestaurantePlato) @Delete fun eliminar(rel: RestaurantePlato) // Eliminar todos los platos de un restaurante @Query("DELETE FROM RestaurantePlato WHERE restaurantId = :restaurantId") fun eliminarPlatosDeRestaurante(restaurantId: Long) }
```

6. Base de datos (todas las entidades)

```
// AppDataBase.kt @Database( entities = [Restaurante::class, Plato::class,
RestaurantePlato::class], version = 1, exportSchema = false ) abstract class AppDataBase :
RoomDatabase() { abstract fun restauranteDao(): RestauranteDao abstract fun platoDao(): PlatoDao
abstract fun restaurantePlatoDao(): RestaurantePlatoDao companion object { @Volatile private var
INSTANCE: AppDataBase? = null fun getDatabase(context: Context): AppDataBase { return INSTANCE
?: synchronized(this) { Room.databaseBuilder( context.applicationContext,
AppDataBase::class.java, "restaurante_db" ) .allowMainThreadQueries() .build() .also { INSTANCE
= it } } } }
```

7. Repository

```
// RestauranteRepository.kt class RestauranteRepository( private val restauranteDao:
RestauranteDao, private val platoDao: PlatoDao, private val relDao: RestaurantePlatoDao ) { fun
insertarRestaurante(r: Restaurante): Long = restauranteDao.insertar(r) fun insertarPlato(p:
Plato): Long = platoDao.insertar(p) // Asociar plato a restaurante fun
asociarPlato(restauranteId: Long, platoId: Long) {
relDao.insertar(RestaurantePlato(restauranteId, platoId)) } fun eliminarRestaurante(r:
Restaurante) = restauranteDao.eliminar(r) fun eliminarPlato(p: Plato) = platoDao.eliminar(p) fun
listarTodo(): LiveData<List<Restaurante>> = restauranteDao.getAll() fun
listarTodoConPlatos(): LiveData<List<RestauranteConPlatos>> =
restauranteDao.getAllConPlatos() fun getRestauranteConPlatos(id: Long): RestauranteConPlatos? =
restauranteDao.getConPlatosById(id) }
```

8. ViewModel

```
// RestauranteViewModel.kt class RestauranteViewModel(application: Application) :
AndroidViewModel(application) { private val repository: RestauranteRepository val
listaConPlatos: LiveData<List<RestauranteConPlatos>> init { val db =
AppDataBase.getDatabase(application) repository = RestauranteRepository( db.restauranteDao(),
db.platoDao(), db.restaurantePlatoDao() ) listaConPlatos = repository.listarTodoConPlatos() } fun
insertarRestaurante(r: Restaurante): Long = repository.insertarRestaurante(r) fun
insertarPlato(p: Plato): Long = repository.insertarPlato(p) fun asociarPlato(restauranteId:
Long, platoId: Long) = repository.asociarPlato(restauranteId, platoId) fun
eliminarRestaurante(r: Restaurante) = repository.eliminarRestaurante(r) fun getConPlatos(id:
Long) = repository.getRestauranteConPlatos(id) }
```

9. Ejemplo de uso en Activity/Fragment

```
// En tu Activity o Fragment: // Insertar restaurante con 3 platos val rId =
viewModel.insertarRestaurante( Restaurante(nombre="La Tasca", telefono="987000000",
foto=R.drawable.tasca, latitud=42.0, longitud=-5.0) ) val p1Id =
viewModel.insertarPlato(Plato(nombre="Cocido maragato")) val p2Id =
viewModel.insertarPlato(Plato(nombre="Botillo")) val p3Id =
viewModel.insertarPlato(Plato(nombre="Cecina")) viewModel.asociarPlato(rId, p1Id)
viewModel.asociarPlato(rId, p2Id) viewModel.asociarPlato(rId, p3Id) // Observar restaurantes con
sus platos viewModel.listaConPlatos.observe(viewLifecycleOwner) { lista -> lista.forEach {
rcp -> Log.d("TAG", "${rcp.restaurante.nombre}: ${rcp.platos.map{it.nombre}}") } }
```

Resumen de decision

	@Embedded	Tabla Intermedia
Platos fijos por restaurante (siempre 3)	✓ Ideal	Posible pero excesivo
Platos variables por restaurante	✗ No escala	✓ Ideal
Un plato en varios restaurantes	✗ Imposible	✓ Native N:M
Buscar restaurantes por plato	✗ Muy difícil	✓ JOIN simple
Simplicidad de implementacion	✓ Alta	Media
Tu caso actual (3 platos fijos)	✓ OK (con correcciones)	Overkill

Correcciones necesarias en tu codigo actual:

1. En AppDataBase: cambia **PlatoDao::class** por **Plato::class** en entities[].
2. El modelo Plato que usas como @Embedded no deberia tener @Entity ni @PrimaryKey — crea un data class separado sin anotaciones de Room para el embedding.
3. Con esas dos correcciones, tu enfoque @Embedded es perfectamente valido para platos fijos.